`schubmult`: SOFTWARE FOR SCHUBERT MULTIPLICATION

MATTHEW J. SAMUEL

ABSTRACT. We present `schubmult`, a Python package for computing entire products of classical and quantum Schubert polynomials, and analyze its elementary-expansion algorithms. The complexity bounds use parameters read from the input: the ambient permutation size n, the lengths r_u, r_v of the trimmed Lehmer codes, and the total inversion number d. A single analysis covers ordinary Schubert multiplication, double Schubert multiplication, the *triple* product $\mathfrak{S}_u(x;y)\mathfrak{S}_v(x;z)$ with independent coefficient alphabets, and the quantum version of each. With $r = \max(r_u, r_v)$ and $h = r_v$, the full triple product uses at most $O(2^{2r}n^{2r+2h+4})$ elementary and coefficient-ring operations. This is $2^{O(n \log n)}$ in general and polynomial in n for fixed code lengths, and it is the same bound we obtain for ordinary products. When u and v are Grassmannian with descent at k, every u-side state of the classical computation is k-Grassmannian.

This uniformity is a feature of the method. It does not follow from specialization, and reducing triple coefficients to ordinary ones monomial by monomial would multiply the cost by the size of the expanded output. The factorial elementary expansion of $\mathfrak{S}_v(x;z)$ is indexed by exactly the same column paths as the ordinary elementary expansion of $\mathfrak{S}_v(x)$. The double Pieri rule is indexed by exactly the same chains as the ordinary one, each carrying a single factorial elementary coefficient. The two extra alphabets therefore enlarge neither the state space nor the label graphs; they cost $O(h^2)$ ring operations per matched pair.

For the quantum triple product, and hence for equivariant and ordinary quantum Littlewood–Richardson coefficients of complete flag varieties, the bound has the same form. It is again $2^{O(n \log n)}$ in general and polynomial in n when r_u, r_v, and the number L of nonzero entries of the code of v^{-1} are fixed. The key observations are that the Fomin–Gelfand–Postnikov quantization commutes with the filtered expansion when column heights are distinct, and that each quantum Pieri step can raise the last descent by at most one, at a cost in q-degree.

Coefficients are represented by arithmetic circuits; no claim about expanded monomial output, symbolic zero testing, or polynomiality in the binary encoding of code entries is made. We compare these full-product bounds with Pak and Slonim's bounded-range algorithm for a single ordinary Schubert coefficient.

1. INTRODUCTION

`schubmult` [21] is a Python package for classical and quantum Schubert calculus. This paper presents its multiplication architecture and analyzes its complexity in terms of the input permutations.

For permutations u, v with finite support, consider the expansion

$$\mathfrak{S}_u(x;y)\mathfrak{S}_v(x;z) = \sum_w c_{u,v}^w(y;z)\mathfrak{S}_w(x;y). \tag{1.1}$$

Here x, y, z are independent infinite alphabets, and the coefficients belong to $\mathbb{Z}[y,z]$. We call (1.1) the *triple* product. At $z = y$ its coefficients are the equivariant Schubert structure constants of complete flag varieties. At $y = z = 0$ they are the ordinary Schubert structure constants, which are nonnegative integers. For $y \neq z$ the coefficients are more general than both. The basic multiplication task of `schubmult` is to compute all the coefficients in (1.1), not just a coefficient specified by a third permutation.

`schubmult` computes such products by matching two computations. An elementary-symmetric expansion of the second factor provides a graph of signed labels. Pieri's rule multiplies the first factor by each elementary factor. Dynamic programming combines these operations without expanding the final coefficients into monomials. The expansion formulas and correctness of this architecture are established in [22, Section 4 and Section 7]. We analyze this architecture through input-only complexity bounds, with the non-quantum algorithm specified precisely enough to account for every search node. We use "classical" only to mean "non-quantum"; it includes the double and triple products.

Date: November 2, 2026.

2020 *Mathematics Subject Classification.* 05E05, 05E14, 14N15, 68Q25.

Key words and phrases. Mathematical software, Schubert polynomial, double Schubert polynomial, Pieri rule, Lehmer code, quantum cohomology, arithmetic complexity.

We use the distinct-position column paths of [22, Definition 4.4 and Theorem 4.5]; their restriction follows from the cancellation theorem proved there. The arithmetic-circuit model and the choice of elementary-weight evaluator are distinguished from literal symbolic implementation costs in Section 8.

Software availability. The package provides Python ring objects and command-line programs, with symbolic arithmetic based on SymEngine and SymPy. It can be installed with `pip install schubmult`. Source code is available at `https://github.com/matthematics/schubmult`, with documentation and usage examples at `https://matthematics.github.io/schubmult/`. Appendix A describes the Grothendieck interfaces, including the conjectural equivariant quantum extension.

Buch's `schubmult` and `lrcalc`. Buch's Littlewood–Richardson Calculator (`lrcalc`) [1] is integrated into SageMath [24]. Its main purpose is the computation of ordinary Grassmannian Littlewood–Richardson coefficients and products of Schur functions. For these tasks its specialized routines are far more efficient in practice than the general Schubert-multiplication methods studied here.

The `lrcalc` distribution also includes Buch's earlier C program `schubmult`, which computes products of ordinary Schubert polynomials and was for many years the standard tool for doing so. It expands one factor into monomials by the transition recursion of Lascoux and Schützenberger [9] and multiplies the monomials into the other factor one variable at a time by Monk's rule [13]. The present Python package is independent and does not include Buch's implementation; its main kernels instead match an elementary-symmetric expansion of one factor against Pieri multiplication of the other, as described below. For comparison, our package contains a separately written implementation of the same transition–Monk algorithm, together with a heuristic choosing between the two methods for ordinary products. These routines are available as Python functions, not through the command-line interface, and are not analyzed here.

1.1. **Examples of use.**

Python ring interface. The ring objects accept permutations in one-line notation. Multiplication returns a Schubert-basis element, whose coefficients can be accessed using `Permutation` keys; `expand()` converts it to an explicit polynomial. Trailing fixed points are omitted in displayed permutation labels.

```
>>> from schubmult import Sx, Permutation
>>> p = Sx([3, 1, 2]) * Sx([1, 3, 2])
>>> p
Sx((3, 2, 1)) + Sx((4, 1, 2, 3))
>>> p[Permutation([3, 2, 1])]
1
>>> p.expand()
x_1**2*x_2 + x_1**3
```

The double ring `DSx` also supports independent coefficient alphabets. Its second argument selects the coefficient alphabet of that factor; omitting it uses y.

```
>>> from schubmult import DSx
>>> from schubmult.abc import z
>>> DSx([1, 3, 2]) * DSx([2, 1, 3], z)
(y_1 - z_1)*DSx((1, 3, 2), y) + DSx((2, 3, 1), y) + DSx((3, 1, 2), y)
```

The quantum counterparts are `QSx` and `QDSx`:

```
>>> from schubmult import QSx, QDSx
>>> QSx([2, 1])**2
q_1 + QSx((3, 1, 2))
>>> QDSx([2, 1])**2
q_1 + (-y_1 + y_2)*QDSx((2, 1), y) + QDSx((3, 1, 2), y)
```

For parabolic quantum multiplication, `QPSx` and its double analogue `QPDSx` take the block sizes. The following computes $\sigma_{1324} \star \sigma_{3412} = q_1 \sigma_{1324}$ in $QH^*(\mathrm{Gr}(2,4))$ and its equivariant counterpart in $QH_T^*(\mathrm{Gr}(2,4))$; `kill_ideal()` imposes the finite-rank quotient after parabolic multiplication.

```
>>> from schubmult import QPSx, QPDSx
>>> A = QPSx(2, 2)
>>> (A([1, 3, 2, 4]) * A([3, 4, 1, 2])).kill_ideal()
q_1*QPSx(2, 2)((1, 3, 2))
```

```
>>> B = QPDSx(2, 2)
>>> (B([1, 3, 2, 4]) * B([3, 4, 1, 2])).kill_ideal()
q_1*QPDSx(2, 2)((1, 3, 2), y) + (-y_1 - y_2 + y_3 + y_4)*QPDSx(2, 2)((3, 4, 1, 2), y)
```

The parabolic quantum ring also supports triple products with independent coefficient alphabets. Passing z to the second factor gives an expansion in the y-basis:

```
>>> (B([3, 4, 1, 2]) * B([1, 3, 2, 4], z)).kill_ideal()
q_1*QPDSx(2, 2)((1, 3, 2), y) + (y_3 + y_4 - z_1 - z_2)*QPDSx(2, 2)((3, 4, 1, 2), y)
```

Command-line tools. Installing the Python package provides the commands below. Permutations are space-separated integers, with a lone `-` separating factors. The ordinary program prints the coefficient followed by the permutation. With `--code`, both input and output use Lehmer codes:

```
$ schubmult_py 3 1 2 - 1 3 2
1  (3, 2, 1)
1  (4, 1, 2, 3)
$ schubmult_py --code 2 - 0 1
1  [2, 1]
1  [3]
```

The double and quantum programs print the permutation first. For the double programs, `--mixed-var` selects independent y, z alphabets; otherwise both factors use y.

```
$ schubmult_double 1 3 2 - 2 1 3 --mixed-var
(1, 3, 2)  y_1 - z_1
(2, 3, 1)  1
(3, 1, 2)  1
$ schubmult_q_double 2 1 - 2 1
()  q_1
(2, 1)  -y_1 + y_2
(3, 1, 2)  1
```

Here `()` denotes the identity permutation. The arguments to `--parabolic` are block sizes, so the parabolic examples above can also be evaluated by

```
$ schubmult_q 1 3 2 4 - 3 4 1 2 --parabolic 2 2
(1, 3, 2)  q_1
$ schubmult_q_double 1 3 2 4 - 3 4 1 2 --parabolic 2 2
(1, 3, 2)  q_1
(3, 4, 1, 2)  -y_1 - y_2 + y_3 + y_4
```

Combining `--parabolic` with `--mixed-var` computes the quantum parabolic triple product:

```
$ schubmult_q_double 3 4 1 2 - 1 3 2 4 --parabolic 2 2 --mixed-var
(1, 3, 2)  q_1
(3, 4, 1, 2)  y_3 + y_4 - z_1 - z_2
```

1.2. **Parameters and main bound.** Permutations are composed as functions, with $(ab)(i) = a(b(i))$. Write

$$\mathfrak{c}_i(w) = \#\{j > i : w(j) < w(i)\}, \qquad \ell(w) = \sum_i \mathfrak{c}_i(w), \qquad r_w = \max\{i : \mathfrak{c}_i(w) > 0\},$$

where $r_{\mathrm{id}} = 0$. Thus r_w is the length of the trimmed Lehmer code and the last descent position of w. We take $n \geq 2$ to be the least integer for which $u, v \in S_n$, apart from this harmless lower bound of 2. It can be recovered from the input codes:

$$n = \max\big(\{2\} \cup \{i + \mathfrak{c}_i(u) : \mathfrak{c}_i(u) > 0\} \cup \{i + \mathfrak{c}_i(v) : \mathfrak{c}_i(v) > 0\}\big).$$

Put $r = \max(r_u, r_v)$, $h = r_v$, and $d = \ell(u) + \ell(v)$. The case $v = \mathrm{id}$ requires no multiplication and will be set aside.

Section 2 constructs a dominant completion with column heights $\theta_1 \geq \cdots \geq \theta_L > 0$. It satisfies $\theta_1 = h$ and $L \leq n-1$. Define the following quantities, all determined or bounded by the input:

$$\begin{aligned} M &= n + L, \\ U &= \min\left\{ (M)_{\underline{r}}, \binom{d+r}{r} \right\}, \\ V &= (n)_{\underline{h}}, \qquad Q_h(n) = \sum_{b=0}^{h} (n-1)_{\underline{b}}. \end{aligned} \tag{1.2}$$

Here $(a)_{\underline{b}} = a!/(a-b)!$ and $(a)_{\underline{0}} = 1$.

Theorem 1.1 (Input-parameter bound)**.** *For $v \neq \mathrm{id}$, Method 4.4 computes the entire triple product* (1.1)*, with coefficient arithmetic circuits, in*

$$O\left(n^3 + LM^3U^2VQ_h(n)\right) \tag{1.3}$$

elementary and coefficient-ring operations. The same bound, with the same proof, holds for the double ($z = y$) and ordinary ($y = z = 0$) products; see Proposition 6.1. In particular, the bound is

$$O\left(2^{2r} n^{2r+2h+4}\right) = 2^{O(n \log n)}. \tag{1.4}$$

For fixed r_u, r_v it is polynomial in n. If v is dominant, the bound improves to $O(n^3 + LM^3U^2)$.

The proof is in Section 6. The exponent in (1.4) is at most $4n \log_2 n + O(n)$.

1.3. **Quantum multiplication.** The same architecture computes the triple quantum product (7.1) of quantum double Schubert polynomials, whose specializations are the equivariant and ordinary quantum Littlewood–Richardson coefficients of complete flag varieties. Two further input parameters enter. Let L be the number of nonzero entries of $\mathfrak{c}(v^{-1})$; it is the number of columns of the completion above. Let $\tilde{h}$ be the largest part of the strict completion (Method 7.1). It is computed from v in $O(n^3)$ operations and, by Lemma 7.2, satisfies

$$\max(r_v, L) \leq \tilde{h} \leq \min(n-1,\ r_v + L - 1).$$

Put $\rho = \max(r_u, \tilde{h})$, $\widetilde{V} = (n)_{\underline{\tilde{h}}}$, and define $R_q \leq 2\rho$ and U_q as in Section 7.

Theorem 1.2 (Quantum input-parameter bound)**.** *For $v \neq \mathrm{id}$, Method 7.4 computes every coefficient $c_{u,v}^{w,\delta}(y;z)$ in* (7.1)*, as arithmetic circuits over $\mathbb{Z}[q][y,z]$, in*

$$O\left(n^3 + LM^3U_q^2\widetilde{V}Q_{\tilde{h}}(n)\right) \tag{1.5}$$

elementary and coefficient-ring operations. Every w that occurs lies in S_{n+L}, and every q^δ that occurs involves only $q_1, \ldots, q_{n+L-1}$; hence the coefficients at $z = y$ are the equivariant quantum Littlewood–Richardson coefficients of Fl_N for every $N \geq n + L$. The bound is

$$O\left(2^{2R_q} n^{2R_q+2\tilde{h}+4}\right) \subseteq O\left(2^{4\rho} n^{6\rho+4}\right) = 2^{O(n \log n)}, \tag{1.6}$$

and it is polynomial in n for fixed r_u, r_v, L. The same bounds hold for the equivariant ($z = y$) and ordinary ($y = z = 0$) quantum products, again with the same state space.

Unlike Theorem 1.1, the polynomial case requires L to be fixed. For $\mathfrak{S}_v = x_1^m$ one has $r_v = 1$ but $\tilde{h} = m$; see Section 7.

1.4. **One bound for ordinary, double, triple, and quantum products.** Theorems 1.1 and 1.2 are stated for the most general products, but the bounds are not weakened by that generality. The ordinary product is not analyzed separately and then lifted. Instead, the triple computation and its ordinary specialization build the same label graphs, make Pieri searches over the same chains, and are bounded by the same state counts. The two coefficient alphabets add only the cost of evaluating one factorial elementary polynomial per matched pair (Proposition 6.1).

This depends on two properties of the method.

(1) The factorial elementary expansion of $\mathfrak{S}_v(x;z)$ (Theorem 3.3) is indexed by the same filtered column paths as the ordinary elementary expansion of $\mathfrak{S}_v(x)$. The alphabet z enters only through the weight attached to each path.

(2) The double Pieri rule (4.1) is indexed by the same canonical chains as Sottile's ordinary Pieri rule. Each chain carries a single factorial elementary polynomial in a fixed alphabet, rather than a sum over additional combinatorial data.

The quantum Pieri rule (7.5) has the same feature, with a q-monomial attached to each chain. Its chains follow the quantum Pieri formula of Postnikov [19] and Ciocan-Fontanine [2] rather than Sottile's rule. Quantum covers add paths: they need not increase length, and the strict completion can be taller than the ordinary one. The extra paths are absorbed by the same counting, with $\tilde{h}$ and R_q in place of h and r, so the asymptotic bound is essentially unchanged (Theorem 1.2).

Neither property is a formal consequence of specialization. Ordinary structure constants do determine the triple coefficients, but the resulting reduction does not give a uniform bound. By Hwang's coefficient lifting [4, Theorem 1.2], the coefficient of y^α in $c^w_{u,v}(y;z)$ is the single coefficient $c^{\hat{w}}_{\hat{u},\hat{v}}(0;z)$ for an explicit block-lifted triple $(\hat{u},\hat{v},\hat{w})$ in $S_{n+|\alpha|}$, where $|\alpha| \le \ell(u)+\ell(v)-\ell(w)$. (Hwang expands in the Schubert basis of the second factor's alphabet, so the two factors of (1.1) are exchanged in his notation; in ours the lifted alphabet y is that of the output basis.) The Cauchy formula then writes the coefficient of each monomial $y^\alpha z^\beta$ as $\pm$ a sum of products of ordinary structure constants [4, Section 5.2]. Thus an algorithm for $c^w_{u,v}$ does yield $c^w_{u,v}(y;z)$, but not parsimoniously. The reduction runs separately for each monomial and each Cauchy term. Each run uses permutations whose rank exceeds n by up to the degree of the monomial. Its cost is therefore multiplied by the expanded size of the triple coefficients, and its input parameters grow. Here, by contrast, the double and triple coefficients come at no extra asymptotic cost because they are produced as arithmetic circuits along the same paths as their ordinary specializations. The circuit representation is essential to this statement, and we make no comparable claim for expanded monomial output.

1.5. **Prior work and the encoding distinction.** Pieri formulas for ordinary Schubert polynomials were established by Sottile [23]; we proved the double Pieri formula used here in [20, Theorem 7.1]. The Cauchy identity and dominant factorization are standard [11]. We studied the triple coefficients in this generality in [20]. Hwang [4] recently showed that each of their monomial coefficients is determined by ordinary structure constants through an explicit block lift; Section 1.4 compares that reduction with the present method. In earlier work [22], we established the ordinary-double expansion [22, Theorem 4.5] and gave the corresponding algorithms [22, Methods 4.9–4.11]. We also established the quantum and doubled Pieri formulas and the quantum expansions [22, Theorems 4.6–4.7], which we use in Section 7. The contribution of the present paper is an explicit accounting of the full-product algorithms in parameters obtainable from their inputs.

Pak and Slonim [18, Theorem 1.4] prove that a *single ordinary* coefficient $c^w_{u,v}$, for three permutations in S_n supplied by their range-restricted codes, can be computed in time

$$\exp\big(O(\psi^{5/2})\big)(\log n)^{O(\psi^2 \log \psi)}. \tag{1.7}$$

Here ψ is their total-range parameter. The bound is polynomial in $\log n$ for fixed ψ. Their method expresses a coefficient as an alternating sum of lattice-point counts and applies Barvinok's algorithm in bounded dimension. The same polyhedral construction proves eventual quasi-polynomiality of stretched Schubert coefficients [18, Theorem 1.1].

Our parameter r measures how far the nonzero Lehmer code extends from its first position. We quote (1.7) with the parameter conventions of its source, rather than deriving a formula relating ψ to r.[1]

There are three important differences between (1.7) and Theorem 1.1. The former returns one integer. The latter returns every coefficient of the triple product, each a polynomial in two alphabets, within the same bound that it gives for the ordinary integer coefficients. The former is a bit-complexity statement for compressed input; the latter initially counts arithmetic and elementary operations. Finally, polynomial dependence on n is much weaker than polynomial dependence on $\log n$. Thus the new bound does not subsume the bounded-range theorem. At the unrestricted extreme our $2^{O(n \log n)}$ bound is smaller in order than the bound obtained by substituting ψ of order n in (1.7); this comparison of upper bounds is not a lower bound on their algorithm or a claim about practical performance.

Encoding also matters for hardness statements. Narayanan proves that Littlewood–Richardson coefficient computation is $\#P$-complete for partitions encoded in binary [15]. The associated Grassmannian Lehmer

[1]The version of [18] consulted defines its range using exceedances $w(i) > i$ and also states that the code vanishes off the exceedance set. These two supports are not generally identical: 321 has code $(2,1,0)$ and exceedance set $\{1\}$. Our parameters and proofs use the Lehmer code directly and do not use that identification.

codes have a polynomial-size binary encoding, so this gives $\#P$-hardness for the correspondingly compressed Schubert problem. It does not automatically give hardness for one-line permutations: writing down a permutation associated with a large binary partition part can require exponentially many entries. Pak and Robichaux discuss this distinction and the unresolved unary-input hardness question [16]. Their subsequent result places Schubert vanishing in probabilistic polynomial time [17], improving the earlier polynomial-hierarchy result. Neither vanishing nor a single-coefficient hardness statement is used in our running-time proof.

2. Dominant completion and a justified label filter

A permutation is *dominant* if its code is a partition. Its Rothe diagram is then a Young diagram. If $\theta = \mathfrak{c}(\tau^{-1})$ is its list of column heights, the dominant factorization is

$$\mathfrak{S}_\tau(x;t) = \prod_{i=1}^{L} \prod_{j=1}^{\theta_i} (x_j - t_i). \tag{2.1}$$

Method 2.1 (Completion)**.** Start with $\mu = v^{-1}$. While its code has a strict adjacent ascent $\mathfrak{c}_i(\mu) < \mathfrak{c}_{i+1}(\mu)$, choose the largest such i and replace μ by μs_i. At termination put $\theta = \mathfrak{c}(\mu)$ with trailing zeros deleted, $\tau = \mu^{-1}$, and $a_* = v\mu$.

Lemma 2.2. *The completion remains in S_n and terminates at a dominant permutation. It satisfies*

$$\ell(a_*) = \ell(\tau) - \ell(v), \qquad \mathrm{Des}(\tau) = \mathrm{Des}(v), \qquad \theta_1 = r_v, \qquad L \le n-1$$

when $v \neq \mathrm{id}$. It can be constructed in $O(n^3)$ elementary operations.

Proof. For an adjacent code ascent, right multiplication by s_i changes the two code entries (a, b) to $(b+1, a)$ and increases length by one. Thus there are at most $\binom{n}{2}$ steps and the final code is weakly decreasing. The successive right multiplications give a length-additive factorization $\mu = v^{-1}a_*$, proving the length identity.

To check the descent assertion, consider the inverses before and after one step. They differ by interchanging the consecutive values $i, i+1$. Their positions are $\mu(i), \mu(i+1)$, with $\mu(i) < \mu(i+1)$. These positions cannot be adjacent: if they were, the two code entries of μ would be equal. Swapping consecutive values in nonadjacent positions does not change any adjacent comparison. The descent set of the inverse therefore stays unchanged throughout. For a dominant permutation the number of nonzero rows of the diagram is its last descent; transposing the diagram identifies this number with θ_1. Finally, the diagram of any permutation in S_n is contained in the staircase, so $L \le n-1$. Maintaining the code, scanning for the next ascent, and updating the permutation costs $O(n)$ per step. □

For $0 \le i \le L$ define the dominant permutation τ_i by

$$\mathfrak{c}(\tau_i^{-1}) = (\theta_1, \ldots, \theta_i), \qquad F_i(x;t) = \mathfrak{S}_{\tau_i}(x;t).$$

Here $\tau_0 = \mathrm{id}$, and $F_i = F_{i-1} \prod_{j=1}^{\theta_i} (x_j - t_i)$. All these permutations are in S_n. Use the signed double-Schubert basis in the auxiliary variables

$$T_a(t;z) = (-1)^{\ell(a)} \mathfrak{S}_a(t;z).$$

We say $\sigma \le_L \tau$ if $\tau = \rho\sigma$ with $\ell(\tau) = \ell(\rho) + \ell(\sigma)$.

Lemma 2.3 (Cauchy filter)**.** *Set*

$$A_i = \{a \in S_n : \ell(a\tau_i) = \ell(\tau_i) - \ell(a)\}.$$

Then

$$F_i(x;t) = \sum_{a \in A_i} \mathfrak{S}_{a\tau_i}(x;z) T_a(t;z). \tag{2.2}$$

Consequently $A_0 = \{\mathrm{id}\}$, $a_* \in A_L$, and the coefficient of T_{a_*} in F_L is $\mathfrak{S}_v(x;z)$. Moreover,

$$|A_i| \le (n)_{\underline{h}} = V. \tag{2.3}$$

Proof. The double Cauchy identity and the interchange-of-alphabets identity give

$$\mathfrak{S}_{\tau_i}(x;t) = \sum_{\substack{\rho\sigma=\tau_i \\ \ell(\rho)+\ell(\sigma)=\ell(\tau_i)}} \mathfrak{S}_\sigma(x;z)\mathfrak{S}_\rho(z;t), \qquad \mathfrak{S}_\rho(z;t) = (-1)^{\ell(\rho)}\mathfrak{S}_{\rho^{-1}}(t;z).$$

Set $a = \rho^{-1}$. Then $\sigma = a\tau_i$ and these are exactly the terms in (2.2). The claims about a_* follow from Lemma 2.2.

For the count, the map $a \mapsto a\tau_i$ is a bijection from A_i onto the left weak interval $[\mathrm{id}, \tau_i]_L$. If $\sigma \le_L \tau_i$, every right descent of σ is a right descent of τ_i: writing $\tau_i = \rho\sigma$, a descent of σ gives $\ell(\tau_i s_j) \le \ell(\rho) + \ell(\sigma) - 1$. The diagram of τ_i has at most h rows, so these permutations have no descent after position h. A permutation in S_n with an increasing suffix after position h is determined by its ordered first h values. There are exactly $(n)_{\underline{h}}$ such permutations. □

The filter is a polynomial identity, not an assertion that each discarded path has zero weight. Coefficients of labels outside A_i can vanish by cancellation. They may be discarded *at a column boundary* because their entire coefficient is zero and subsequent multiplication is linear. In particular, this does not justify pruning the interior of a column merely because an intermediate path node fails the boundary test.

3. A column expansion with a direct proof

For an ordered alphabet $Y = (Y_1, \dots, Y_m)$ define

$$E_{p,m}(Y;Z) = \sum_{1 \le j_1 < \cdots < j_p \le m} \prod_{q=1}^{p} (Y_{j_q} - Z_{j_q - q + 1}), \qquad 0 \le p \le m. \tag{3.1}$$

Only the first $m - p + 1$ entries of Z are used when $p > 0$. Set $E_{0,m} = 1$ and $E_{p,m} = 0$ for $p > m$. Repeated entries in Z are allowed.

A *pivot-i path* of length b is a sequence $P = (a_0, \dots, a_b)$ with

$$a_j = a_{j-1} t_{i,c_j}, \qquad c_j \ne i, \qquad \ell(a_j) = \ell(a_{j-1}) + 1.$$

No distinctness condition is imposed on the c_j. Associate to it

$$\varepsilon(P) = (-1)^{\#\{j : c_j < i\}}, \qquad Z(P) = (z_{a_0(i)}, z_{a_1(i)}, \dots, z_{a_b(i)}).$$

Lemma 3.1 (Unrestricted column formula)**.** *For $C_i = \prod_{j=1}^{k}(x_j - t_i)$,*

$$C_i T_a(t;z) = \sum_{b=0}^{k} \sum_{P : a_0 = a} \varepsilon(P) E_{k-b,k}(x_1, \dots, x_k; Z(P)) T_{a_b}(t;z). \tag{3.2}$$

Proof. The equivariant single-variable Monk formula, in the signed basis, is

$$(X - t_i)T_a = (X - z_{a(i)})T_a + \sum_{\substack{j > i \\ \ell(a t_{ij}) = \ell(a)+1}} T_{a t_{ij}} - \sum_{\substack{j < i \\ \ell(a t_{ji}) = \ell(a)+1}} T_{a t_{ji}}. \tag{3.3}$$

It follows by subtracting the formulas for multiplication by $t_1 + \cdots + t_i$ and $t_1 + \cdots + t_{i-1}$. Apply (3.3) successively with $X = x_1, \dots, x_k$. Fix the subsequence of non-diagonal steps, namely a path P of length b. Its sign is $\varepsilon(P)$. Choose the $k - b$ diagonal positions $j_1 < \cdots < j_{k-b}$ among the k factors. Before the diagonal position j_q there have been $j_q - q$ non-diagonal steps, so its contribution is $x_{j_q} - z_{a_{j_q - q}(i)}$. Summing over the diagonal positions gives exactly $E_{k-b,k}(x; Z(P))$ in (3.1). □

Lemma 3.2 (Distinct-position cancellation)**.** *The sum in* (3.2) *is unchanged if it is restricted to paths whose nonpivot positions $c_1, \dots, c_b$ are pairwise distinct. For these paths $Z(P)$ consists of the values*

$$D_i(a_0, a_b) = \{a_0(i)\} \cup \{a_0(j) : a_0(j) \ne a_b(j)\},$$

each once, and its order is immaterial.

Proof. This is the cancellation underlying [22, Lemma 7.5 and Theorem 7.6]. We explain how it applies to the signed-basis convention used here. In a path with a repetition, take the segment ending at the first repeated nonpivot position, starting at its earlier occurrence. The rotation lemma pairs its valid cyclic rotations with starting positions on opposite sides of the pivot. The resulting involution fixes the two endpoints, the path length, and the multiset of values encountered at the pivot, and reverses $\varepsilon(P)$. By [22, Lemma 7.3], $E_{k-b,k}(x;Z)$ is symmetric in the $b+1$ entries of Z. The two weights therefore cancel. The sign convention in the cited skew-divided-difference formula differs by $(-1)^b$, which is constant on each such pair.

On a distinct-position path the pivot successively carries $a_0(i), a_0(c_1), \dots, a_0(c_b)$. These are exactly the values in $D_i(a_0, a_b)$. The same symmetry gives the last assertion. □

Theorem 3.3 (Filtered elementary expansion)**.** *Start at label* id *and, for $i = 1, \dots, L$, take a pivot-i path P_i of length $b_i \le \theta_i$ with pairwise distinct nonpivot positions within that path. Its starting label is the previous path's endpoint. Require the endpoint at column i to belong to A_i and the final endpoint to be a_*. Then*

$$\mathfrak{S}_v(x;z) = \sum_{(P_1,\dots,P_L)} \prod_{i=1}^{L} \varepsilon(P_i) E_{\theta_i - b_i, \theta_i}(x; Z(P_i)). \tag{3.4}$$

Proof. Apply Lemmas 3.1 and 3.2 to the factors of F_L. At each column boundary, Lemma 2.3 permits deleting all labels outside A_i. Extracting T_{a_*} gives (3.4). A path reaching $a_* \in S_n$ never leaves S_n, since all its steps increase Bruhat order and the interval below a permutation in S_n is contained in S_n. Before imposing the boundary filters, this is precisely [22, Theorem 4.5]. □

Setting $z = 0$ in (3.4) replaces $E_{p,m}$ by $e_p(x_1, \dots, x_m)$ and preserves the paths and their signs. This is a statement about the *path-indexed* expansion. It is not a way of recovering the double polynomial from a collected ordinary expansion after its path labels and alphabets have been erased. No uniqueness or minimality of an elementary representation is needed.

Method 3.4 (Backward construction of the label graphs)**.** Start with the singleton set $\{a_*\}$ at boundary L. For $i = L, L-1, \dots, 1$ and each currently retained upper label, enumerate all descending pivot-i Bruhat paths of lengths $0, \dots, \theta_i$, never reusing a nonpivot position in a path. Retain a path if its lower endpoint is in A_{i-1}. Reverse it to obtain a forward edge, storing its two endpoints, length, sign, and ordered alphabet indices. Collect the distinct lower endpoints to seed the next boundary. Do not prune internal path nodes using the endpoint filter.

This construction gives exactly the paths in Theorem 3.3 that can reach the required final label; it does not enumerate every permutation in A_i. Each stored edge represents an entire column path, not a single Bruhat cover. The product of distinct pivot transpositions is a cycle, from which their order can be recovered, so at most one such path has a specified pair of endpoints. This is the `kdown/vpaths` construction of [22, Methods 4.9–4.10]. In its implementation, a nonpivot position is eligible exactly when it still holds its value from the start of the path: distinct pivot swaps change every position already used and leave all others untouched.

4. Pieri multiplication and coefficient construction

For a permutation w and an integer k, a canonical elementary Pieri chain consists of Bruhat covers $w_j = w_{j-1} t_{a_j b_j}$ with $a_j \le k < b_j$, whose moved-in values $w_{j-1}(b_j)$ are strictly decreasing. The elementary Pieri rule identifies chains of length a with the terms of $e_a(x_1, \dots, x_k)\mathfrak{S}_w(x)$, each with multiplicity one [23]. In particular, distinct canonical chains from a fixed w have distinct endpoints, including across different lengths. No left position can be used twice: a second ascent there would require a moved-in value greater than the first, contrary to the ordering.

Write $w \longrightarrow_k w'$ for such an endpoint and let $a = \ell(w') - \ell(w)$. The fixed alphabet is

$$Y(w, w'; k) = (y_{w(j)} : 1 \le j \le k,\ w(j) = w'(j)),$$

listed in increasing order of j; it has $k - a$ entries. The double Pieri formula [20, Theorem 7.1] is

$$\mathfrak{S}_w(x;y) E_{p,k}(x;Z) = \sum_{\substack{w \longrightarrow_k w' \\ a \le p}} E_{p-a,k-a}(Y(w,w';k); Z) \mathfrak{S}_{w'}(x;y). \tag{4.1}$$

Since this is a polynomial identity in the coefficient alphabet, it remains valid when entries of Z coincide, as they may in Lemma 3.1.

Method 4.1 (Pieri enumeration)**.** Maintain a frontier (w, β), initially $(w, +\infty)$. To extend it, test $a \le k < b$ for

$$w(a) < w(b) < \beta, \qquad \nexists c\ (a < c < b,\ w(a) < w(c) < w(b)).$$

For each successful test append $(wt_{ab}, w(b))$ to the next frontier. Return the initial permutation and every endpoint through the requested maximum degree $p \le k$, recording its degree.

Proposition 4.2. *If all generated permutations lie in S_M and B endpoints are returned, Method 4.1 costs $O(M^3 B)$ elementary operations.*

Proof. A frontier node has at most $k(M-k)$ candidate transpositions. Testing the Bruhat-cover condition and copying a permutation each cost $O(M)$. Multiplicity-freeness of the canonical Pieri chains bounds the total number of processed nodes by the number of returned endpoints. □

Lemma 4.3 (Polynomial-cost elementary weights)**.** *A factor $E_{p,m}(Y; Z)$ can be built as an arithmetic circuit using $O((m+1)^2)$ ring operations.*

Proof. Separating subsets according to whether they contain the last index gives

$$E_{p,m}(Y; Z) = E_{p,m-1}(Y_1, \dots, Y_{m-1}; Z) + (Y_m - Z_{m-p+1}) E_{p-1,m-1}(Y_1, \dots, Y_{m-1}; Z). \tag{4.2}$$

Use $E_{0,t} = 1$ and $E_{q,t} = 0$ for $q > t$, and memoize the subproblems (q, t) reached from (p, m). There are at most $(m+1)^2$ of them, each costing a bounded number of operations. Every nontrivial reached subproblem has $t - q \le m - p$, so no coefficient variable beyond Z_{m-p+1} is requested. The circuit shares subexpressions rather than copying expression trees. □

An unmemoized divide-and-conquer evaluation is another valid choice: split Y after $s = \lfloor m/2 \rfloor$ and use

$$E_{p,m}(Y; Z) = \sum_a E_{a,s}(Y'; Z) E_{p-a,m-s}(Y''; Z^{[s-a]}), \qquad Z_j^{[c]} = Z_{j+c},$$

where the sum is over feasible degrees. Its recursion gives an upper bound $m^{O(\log m)}$. That is an evaluation-strategy cost, not an intrinsic lower bound for factorial elementary polynomials. Theorem 1.1 uses (4.2).

Method 4.4 (Classical `schubmult`)**.** Construct the completion and the label graphs. The coefficient ring is $\mathbb{Z}[y, z]$. Initialize a table H with $H(u, \mathrm{id}) = 1$. At column i put $k = \theta_i$ and create a fresh table H'.

(1) Group the keys of H by their first component w. For each distinct w enumerate its Pieri endpoints once, through degree $\min(k, d - \ell(w))$. Reuse this list for all labels paired with w.
(2) For each table entry $H(w, a)$, each label edge $P : a \to a'$ of length b, and each Pieri endpoint w' of degree a_u, use the pair only if $a_u + b \le k$. Accumulate

$$H'(w', a') \mathrel{+}= H(w, a)\varepsilon(P) E_{k-a_u-b,k-a_u}\big(Y(w, w'; k); Z(P)\big).$$

(3) Replace H by H' and continue. After column L, return $H(w, a_*)$ as the coefficient of $\mathfrak{S}_w(x; y)$.

In the ordinary specialization keep only $a_u + b = k$; the elementary weight is then 1.

Theorem 4.5 (Correctness)**.** *Method 4.4 computes* (1.1).

Proof. Multiply the expansion in Theorem 3.3 by $\mathfrak{S}_u(x; y)$ and apply (4.1) at each column. The displayed update is exactly the product of the column sign, the running coefficient, and the Pieri coefficient. Dynamic programming merely combines contributions with equal endpoint pairs. The final label condition is precisely that in Theorem 3.3.

It remains to justify the degree cap. Along a complete label path, $\sum_i b_i = \ell(a_*) = \ell(\tau) - \ell(v)$ and $\sum_i \theta_i = \ell(\tau)$. Every admissible paired path has $\sum_i a_{u,i} \le \sum_i (\theta_i - b_i) = \ell(v)$. All $a_{u,i}$ are nonnegative, so no contributing prefix can reach a permutation of length greater than d. Thus the cap discards no contribution. Specialization to $y = z = 0$ annihilates every elementary weight of positive degree and leaves precisely the stated ordinary update. □

5. Input bounds for every enumerated state

Lemma 5.1 (Pieri ambient size)**.** *If $w \in S_m$ and $k < m$, a canonical elementary Pieri chain from w stays in S_{m+1}.*

Proof. Before any position beyond m is used, a Bruhat cover cannot have right endpoint $b \geq m+2$: position $m+1$ has value $m+1$, strictly between $w(a) \leq m$ and $w(b) = b$. The only possible new value is therefore $m+1$. If it is moved in, it must be the first moved-in value, since all previously available values are at most m and the sequence decreases. All subsequent moved-in values are smaller; no larger new position can be used. □

Lemma 5.2 (Code-length and degree bounds)**.** *Every u-side permutation enumerated by Method 4.4, including nodes of Pieri searches that do not survive into the next table, lies in S_M, has no descent after r, and has length at most d. There are at most U such permutations.*

Proof. The ambient claim follows by applying Lemma 5.1 once per column, starting in S_n. The degree cap in Method 4.4 bounds every partial Pieri chain by d.

For the descent claim, let w have no descent after r. Since $k \leq h \leq r$, the polynomial

$$e_a(x_1, \ldots, x_k)\mathfrak{S}_w(x)$$

involves only $x_1, \ldots, x_r$. For $j > r$ its divided difference ∂_j is zero. For a Schubert basis element, $\partial_j \mathfrak{S}_b = \mathfrak{S}_{bs_j}$ if $j \in \mathrm{Des}(b)$; otherwise its divided difference is zero. Distinct b with descent j give distinct bs_j. Linear independence of the Schubert basis therefore implies that no permutation in this product has a descent after r. Every canonical chain of length a is a term of this ordinary Pieri product with coefficient one, so this argument applies to every enumerated endpoint, before any double-coefficient cancellation. Induction starts with u.

Permutations of S_M with an increasing suffix after r number $(M)_{\underline{r}}$. Alternatively, their Lehmer codes have length at most r and sum at most d. There are $\binom{d+r}{r}$ nonnegative r-tuples with that sum bound, by stars and bars. Taking the smaller estimate proves the assertion. □

Lemma 5.3 (Cost of the label construction)**.** *Method 3.4 stores at most $LVQ_h(n)$ column paths and costs $O(n^3 + Ln^3VQ_h(n))$ elementary operations. This estimate charges paths rejected by the endpoint filter.*

Proof. There are at most V retained upper labels at a boundary, by Lemma 2.3. From each upper label, a descending pivot path has depth at most $\theta_i \leq h$ and uses each nonpivot position at most once. There are at most $(n-1)_{\underline{b}}$ possible position sequences of length b, even before testing the Bruhat conditions. The entire search tree, including rejected nodes, consequently has at most $Q_h(n)$ nodes. Every node is in S_n. Scanning candidates, testing Bruhat covers, computing the endpoint length test, and recording a path take polynomial work bounded by $O(n^3)$ per node. Summing over boundaries and upper labels proves the claim. Constructing the prefixes τ_i and the completion fits the additional $O(n^3)$ term. □

6. Complexity and its interpretation

6.1. **Arithmetic model.** A coefficient is stored as a directed acyclic arithmetic circuit over $\mathbb{Z}[y, z]$; addition and multiplication create gates pointing to existing gates. The elementary routine shares its dynamic-programming subexpressions. We count these ring operations and elementary operations on permutation entries separately from bit costs. Permutation entries have $O(\log M)$ bits. Deterministic dictionaries may be implemented by balanced trees: comparing two keys costs $O(M)$ elementary operations, and a table has at most UV keys. Since $\log(UV) = O(M \log M)$, this bookkeeping is absorbed by the M^3 factor below. No unit-cost polynomial identity test is assumed.

The returned table is an exact expansion but need not be a minimal-support list. An identically zero coefficient can be left as a circuit computing zero. Removing all such entries, expanding coefficients into monomials, and canonical symbolic simplification are different output requirements and are not included in the theorem.

Proof of Theorem 1.1. The label preprocessing costs $O(n^3 + Ln^3VQ_h(n))$ by Lemma 5.3. At a product boundary there are at most U distinct u-side permutations and at most V labels. At one column the Pieri enumeration is called at most U times, each returning at most U endpoints by Lemma 5.2. Proposition 4.2 bounds this work by $O(M^3U^2)$.

Count matched pairs by iterating over all column edges, rather than assuming a bound on the forward outdegree of each label. There are at most $VQ_h(n)$ such edges in a column. For each there are at most U possible source permutations, and each source has at most U Pieri endpoints. Thus there are at most $U^2VQ_h(n)$ pairs. Forming the fixed alphabet takes $O(M)$ work. Lemma 4.3 builds its elementary coefficient in $O((h+1)^2)$ ring operations; the remaining update takes a bounded number of ring operations. Permutation and dictionary work, as well as this arithmetic, is bounded by $O(M^3)$ per pair. Summing over L columns and absorbing the preprocessing and Pieri terms gives (1.3).

For the coarse estimate use

$$L \le n-1, \quad M \le 2n-1, \quad U \le (2n)^r, \quad V \le n^h, \quad Q_h(n) \le 2n^h.$$

The last bound follows by comparing $(n-1)_{\underline{b}}$ with $(n-1)^b$ and summing the geometric series for $n \ge 3$, and for $n = 2$ because $h \le 1$. Substitution gives (1.4). Since $h \le r \le n-1$, its logarithm is at most $4n \log_2 n + O(n)$.

If v is dominant, $\mu = v^{-1}$ and $a_* = \mathrm{id}$. No descending path of positive length starts at the identity. Every label graph therefore consists of the single degree-zero edge $\mathrm{id} \to \mathrm{id}$, and the entire product calculation has one label. Replacing $VQ_h(n)$ by 1 gives the dominant-input estimate. By Proposition 6.1, the double and ordinary products satisfy the same bound. □

6.2. **Uniformity across coefficient rings.** The ordinary specialization of Method 4.4 keeps only pairs with $a_u + b = k$. In the double specialization, set $z = y$.

Proposition 6.1 (Uniformity)**.** *Fix u, v with $v \neq \mathrm{id}$. Compare the triple computation of Method 4.4 with its double and ordinary specializations.*

1. *All three use the same completion and the same label graphs.*
2. *Every table key and every Pieri call made by the ordinary computation is made by the triple computation, with the same arguments. Every matched pair used by the ordinary computation is used by the triple computation. The triple computation's additional pairs are those with $a_u + b < k$.*
3. *Per matched pair, the triple and double computations perform $O((h+1)^2)$ ring operations and $O(M)$ elementary operations beyond the ordinary computation.*

In particular, all three computations satisfy (1.3)*. The same statements hold for Method 7.4, with $t + b = k$ in place of $a_u + b = k$ and $\tilde{h}$ in place of h, except that a* `qpieri` *call of the triple computation may have a larger depth p_w. Its returned list then contains that of the ordinary call.*

Proof. The completion and the label graphs depend only on v. The recorded alphabet indices $Z(P)$ are used only to form weights, never to accept or reject a path. Step (1) of Method 4.4 depends only on w, k, and d. Step (2) accepts a pair according to a_u and b alone. By induction over columns, the ordinary keys at each boundary therefore form a subset of the triple keys, and the ordinary pairs form a subset of the triple pairs. The additional work for a triple pair is the formation of $Y(w, w'; k)$ and the circuit of Lemma 4.3. In the ordinary computation the weight is 1. The proofs of Lemmas 2.3, 5.2, and 5.3 never use the coefficient ring, so they apply to all three computations. For the quantum method, the same argument applies with Lemmas 7.2 and 7.7. A `qpieri` search of depth p is a prefix of one of larger depth, and Lemma 7.7 bounds every chain that the triple computation enumerates. □

The proposition is an upper-bound statement. The ordinary computation can be cheaper in practice, since it discards pairs with $a_u + b < k$. The point is that the worst-case analysis does not distinguish the three problems, and the bound for the triple product is not obtained by paying for its extra generality.

Corollary 6.2 (Ordinary bit costs)**.** *For unit initial coefficient, the ordinary algorithm has bit complexity bounded by* (1.3) *times a polynomial in n. It therefore still has complexity $2^{O(n \log n)}$, and polynomial complexity in n for fixed r_u, r_v.*

Proof. Each surviving matched pair contributes the running integer with sign $+1$ or -1. The absolute value of any intermediate sum is bounded by the number of complete or partial paired paths. A state has at most $UVQ_h(n)$ successors counted with multiplicity, so that number is at most $(2UVQ_h(n))^L$. Its bit length is

$$O\big(1 + L \log(2UVQ_h(n))\big) = O(n^2 \log n).$$

Integer additions and permutation operations consequently incur only a polynomial bit overhead. This argument bounds intermediate signed sums as well as final nonnegative coefficients; it does not rely on positivity preventing intermediate cancellation. □

6.3. **Dependence on code length.** The number of code entries is a useful parameter because the u-side count can be replaced by $\binom{d+r}{r}$ when degree is more restrictive than the ambient permutation size. No weak-order interval enumeration is needed for this refinement. The v-side bound depends on $h = r_v$, reflecting the fact that the second factor supplies the elementary expansion. For ordinary multiplication the roles may be interchanged and the smaller of the two bounds used. For (1.1), such an interchange also changes which coefficient alphabet indexes the output basis and is not automatic.

An input code with r_w entries has encoding length $O(r_w \log n)$, whereas the algorithm explicitly works with permutations and has polynomial dependence on n for fixed r. This is a pseudo-polynomial feature relative to binary code entries. For example, $\mathfrak{S}_{\mathrm{uncode}(0,A)}(x_1, x_2) = h_A(x_1, x_2)$, and

$$h_A(x_1, x_2)^2 = \sum_{j=0}^{A} s_{(2A-j,j)}(x_1, x_2).$$

The two input codes have constant length and $O(\log(A + 1))$ bits, but an explicit Schubert-basis list has $A + 1$ terms.

Under scaling of the entries of fixed Lehmer codes by a factor N, the code lengths remain fixed and $n = O(N)$. Theorem 1.1 hence gives a polynomial-in-N bound for these full products. This does not yield the polylogarithmic dependence available from (1.7) in its fixed-range regime, nor does it prove eventual quasi-polynomiality.

Corollary 6.3 (Grassmannian inputs)**.** *Let $u, v \in S_n$, $v \neq \mathrm{id}$, have no descents other than at k. Then every column of the completion has height k, and every u-side permutation enumerated by Method 4.4, including Pieri search nodes, is k-Grassmannian. Every label $a \in A_i$ has $a\tau_i$ k-Grassmannian. Consequently U and V in* (1.3) *may be replaced by*

$$U_k = \min\left\{\binom{M}{k}, \binom{d+k}{k}\right\}, \qquad V_k = \binom{n}{k},$$

giving

$$O\left(n^3 + LM^3U_k^2V_kQ_k(n)\right) = O\left(n^4\binom{2n}{k}^2\binom{n}{k}n^k\right).$$

Uniformly in k this is $O(4^k n^{4k+4}/(k!)^3)$, and in general it is $2^{O(n)}n^{k+4}$.

Proof. By Lemma 2.2, $\mathrm{Des}(\tau) = \mathrm{Des}(v) = \{k\}$. A dominant permutation whose only descent is at k has a rectangular diagram with k rows, so its transpose θ has all parts equal to k. For the labels, $a\tau_i \leq_L \tau_i$, so as in the proof of Lemma 2.3 its descents lie in $\mathrm{Des}(\tau_i) = \{k\}$; there are at most $\binom{n}{k}$ such permutations in S_n.

Let w be k-Grassmannian. Then $\mathfrak{S}_w(x)$ is symmetric in $x_1, \ldots, x_k$ and involves no other x-variables, so the same holds for $e_a(x_1, \ldots, x_k)\mathfrak{S}_w(x)$. Hence ∂_j annihilates it for every $j \neq k$. The argument of Lemma 5.2 shows that every permutation in this ordinary Pieri product, and so every canonical chain endpoint, has no descent other than at k. Induction from u covers all enumerated states, including those of the double computation, whose Pieri chains are the same. A k-Grassmannian permutation in S_M is determined by its first k values, as a set, giving $\binom{M}{k}$. Alternatively, its code is a partition with at most k parts, read in reverse, and of size at most d; there are at most $\binom{d+k}{k}$ of these. The displayed estimate follows from the proof of Theorem 1.1, with $M \leq 2n$ and $Q_k(n) \leq 2n^k$. □

Thus Grassmannian multiplication works entirely with partitions in a $k \times (M - k)$ box on the u-side, with $L = \mathfrak{c}_k(v) \leq n - k$ columns. The remaining non-exponential factor n^k comes from the label search $Q_k(n)$, which counts rejected distinct-position paths. Sharpening that count would make the Grassmannian bound $2^{O(n)}$.

7. Quantum multiplication

We now prove Theorem 1.2, directly for the triple quantum product. The architecture is unchanged: a filtered elementary expansion of the second factor is paired with a Pieri rule for the first. Two ingredients change. The factors must be quantized one column at a time, which forces pairwise distinct column heights; and the quantum Pieri rule has length-decreasing steps, which requires new state estimates.

7.1. **Quantization and the strict completion.** Let $X_j = (x_1, \dots, x_j)$. A *standard elementary monomial* is a product $\prod_j e_{c_j}(X_j)$ in which each height j occurs at most once and $0 \le c_j \le j$. These monomials form a $\mathbb{Z}$-basis of $\mathbb{Z}[x]$. The quantization map of Fomin, Gelfand, and Postnikov [3] is the $\mathbb{Z}[q]$-linear map, extended linearly over all coefficient alphabets, sending $\prod_j e_{c_j}(X_j)$ to $\prod_j e^q_{c_j}(X_j)$, where $e^q_c(X_j)$ are the quantum elementary polynomials. It sends $\mathfrak{S}_w(x;y)$ to the quantum double Schubert polynomial $\widetilde{\mathfrak{S}}_w(x;y)$ [3, 7], [22, Lemma 2.1]. These polynomials, indexed by S_∞, form a $\mathbb{Z}[q][y]$-basis of $\mathbb{Z}[q][x,y]$, so we may write

$$\widetilde{\mathfrak{S}}_u(x;y)\widetilde{\mathfrak{S}}_v(x;z) = \sum_{w,\delta} q^\delta c^{w,\delta}_{u,v}(y;z)\,\widetilde{\mathfrak{S}}_w(x;y). \tag{7.1}$$

At $z = y$ the coefficient $c^{w,\delta}_{u,v}(y;y)$ is the equivariant quantum Littlewood–Richardson coefficient of Fl_N for every N such that every occurring w lies in S_N and every occurring degree has $\delta_i = 0$ for $i \ge N$ [6, 8, 12], [22, Section 2]. At $y = z = 0$ it is the ordinary quantum structure constant. Grading $\deg q_i = 2$ makes (7.1) homogeneous. Put

$$\widetilde{E}_{p,k}(x;Z) = \mathcal{Q}\big(E_{p,k}(x;Z)\big),$$

where $\mathcal{Q}$ is the quantization map. Expanding the definition, $E_{p,k}(x;Z)$ is a $\mathbb{Z}[Z]$-linear combination of $e_c(X_k)$ with $c \le k$, so $\widetilde{E}_{p,k}$ is the corresponding combination of $e^q_c(X_k)$ [22, Lemma 7.3].

The classical completion of Section 2 usually has repeated column heights, and $\mathcal{Q}$ is not multiplicative on $e_a(X_k)e_b(X_k)$. We therefore use the strict completion of [22, Definition 4.1]. Call μ *strict dominant* if it is dominant and its nonzero code entries are strictly decreasing.

Method 7.1 (Strict completion)**.** Start with $\mu = v^{-1}$. While μ is not strict dominant, choose the largest i with $\mathfrak{c}_{i+1}(\mu) \ne 0$ and $\mathfrak{c}_i(\mu) \le \mathfrak{c}_{i+1}(\mu)$, and replace μ by μs_i. At termination put $\tilde\theta = \mathfrak{c}(\mu)$ with trailing zeros deleted, $\tilde\tau = \mu^{-1}$, $\tilde a_* = v\mu$, and $\tilde h = \tilde\theta_1$.

Lemma 7.2 (Strict height)**.** *Let $v \ne \mathrm{id}$ and let L be the number of nonzero entries of $\mathfrak{c}(v^{-1})$. The strict completion stays in S_n, uses $O(n^3)$ elementary operations, and satisfies $\ell(\tilde a_*) = \ell(\tilde\tau) - \ell(v)$. Its shape has exactly L parts, $\tilde\theta_1 > \cdots > \tilde\theta_L > 0$, and the classical completion also has exactly L columns. Finally,*

$$\max(r_v, L) \le \tilde h \le \min\big(n-1,\ r_v + L - 1\big). \tag{7.2}$$

Proof. A step at i has $\mathfrak{c}_i(\mu) \le \mathfrak{c}_{i+1}(\mu)$, so $\mu(i) < \mu(i+1)$; it replaces the entries (a,b) by $(b+1,a)$ and increases length by one. Since $b \ne 0$, we have $i+1 \le m-1$ whenever $\mu \in S_m$, so $\mu s_i \in S_m$. This proves the ambient claim for $m = n$, termination, the length identity, and the $O(n^3)$ cost as in Lemma 2.2. The replacement $(a,b) \mapsto (b+1,a)$ with $b \ne 0$ preserves the number of nonzero entries. The same holds for the classical steps, which have $a < b$. Hence both shapes have L parts, and the termination condition gives strictly decreasing parts.

A strictly decreasing sequence of L positive integers has first entry at least L. For the other lower bound, $\tilde\tau = x^{-1}v$ is length-additive, so, as in the proof of Lemma 2.3, every right descent of v is one of $\tilde\tau$. The permutation $\tilde\tau$ is the inverse of a dominant permutation, hence dominant, and its last descent is the number of rows of its diagram, namely $\tilde\theta_1$. Thus $r_v \le \tilde h$.

For the upper bound let μ be the classical completion, with shape θ, and put $m = \max_j(\theta_j + j)$. A dominant permutation with shape θ lies in S_m: its code satisfies $\mathfrak{c}_j \le m - j$. Since $\mu = v^{-1}x$ with lengths adding, v^{-1} is a prefix of a reduced word of μ and so lies in S_m. Applying the first paragraph with this m shows that the strict completion lies in S_m, whence $\tilde h = \mathfrak{c}_1 \le m - 1$. Finally, $\theta_j \le \theta_1 = r_v$ and $j \le L$ give $m \le r_v + L$. The bound $\tilde h \le n-1$ is the case $m = n$. □

Both inequalities in (7.2) can be sharp. For $v = (m+1)12\cdots m$, which has $\mathfrak{S}_v = x_1^m$, one has $r_v = 1$, $L = m$, $\theta = (1^m)$, and $\tilde\theta = (m, m-1, \dots, 1)$. Computations through S_8 also gave $\tilde h = \max_j(\theta_j + j) - 1$ for every nonidentity v.

7.2. **Quantum filtered expansion.** Define $\tilde{\tau}_i$, $\tilde{F}_i$, $\tilde{A}_i$, and the label graphs exactly as in Sections 2 and 3, with $\tilde{\theta}$ in place of θ, and put

$$\widetilde{V} = (n)_{\underline{\tilde{h}}}.$$

The proofs of Lemmas 2.3 and 5.3 use only that the completion is dominant, lies in S_n, and satisfies the length identity, and that its first column has height equal to the number of rows of the diagram of each $\tilde{\tau}_i$. They therefore give $|\tilde{A}_i| \le \widetilde{V}$, and a label construction with at most $L\widetilde{V}Q_{\tilde{h}}(n)$ edges costing $O(n^3 + Ln^3\widetilde{V}Q_{\tilde{h}}(n))$. Likewise, the proof of Theorem 3.3 gives

$$\mathfrak{S}_v(x;z) = \sum_{(P_1,\dots,P_L)} \prod_{i=1}^{L} \varepsilon(P_i) E_{\tilde{\theta}_i - b_i, \tilde{\theta}_i}(x; Z(P_i)), \tag{7.3}$$

summed over the filtered distinct-position paths ending at $\tilde{a}_*$.

Theorem 7.3 (Filtered quantum expansion)**.** *With the same index set as* (7.3)*,*

$$\widetilde{\mathfrak{S}}_v(x;z) = \sum_{(P_1,\dots,P_L)} \prod_{i=1}^{L} \varepsilon(P_i) \widetilde{E}_{\tilde{\theta}_i - b_i, \tilde{\theta}_i}(x; Z(P_i)). \tag{7.4}$$

Proof. Apply $\mathcal{Q}$ to (7.3). On the left this gives $\widetilde{\mathfrak{S}}_v(x;z)$. Each factor in a summand is a $\mathbb{Z}[z]$-combination of $e_c(X_{\tilde{\theta}_i})$, and the heights $\tilde{\theta}_i$ are pairwise distinct. Expanding a summand therefore gives a $\mathbb{Z}[z]$-combination of standard elementary monomials. Hence $\mathcal{Q}$ maps the summand to the product of the quantized factors. □

Without the filters, (7.4) is [22, Theorem 4.6]. The proof shows more: for *any* set of complete distinct-height column paths, the sum of quantized weights is the quantization of the sum of classical weights. Thus every boundary filter justified classically is automatically justified in quantum cohomology, without a new cancellation argument.

7.3. **The quantum Pieri step.** A *quantum cover* of w is a transposition t_{ab}, $a < b$, with $\ell(wt_{ab}) = \ell(w)+1$, or with $w(a) > w(b)$ and $\ell(wt_{ab}) = \ell(w) + 1 - 2(b-a)$. The latter is equivalent to every $w(c)$, $a < c < b$, lying strictly between $w(b)$ and $w(a)$, and it carries the weight $q_a \cdots q_{b-1}$. The routine `qpieri`(w, p, k) of [22, Method 4.12] is Method 4.1 with quantum covers in place of Bruhat covers, with $a \le k < b$, with each left position used at most once, and with the right positions b weakly decreasing along a chain. It returns triples $(w', t, q^{\delta'})$, one for every node of its search tree. By [22, Lemma 4.13] it returns each endpoint once and, for $p' \le p \le k$,

$$\widetilde{\mathfrak{S}}_w(x;y)\widetilde{E}_{p',k}(x;Z) = \sum_{\substack{(w',t,q^{\delta'}) \\ t \le p'}} q^{\delta'} E_{p'-t,k-t}\big(Y(w,w';k); Z\big)\widetilde{\mathfrak{S}}_{w'}(x;y). \tag{7.5}$$

Here t is the number of transpositions in the chain, and $Y(w,w';k)$ is as in Section 4. Since left positions are used once, it has $k-t$ entries. Multiplicity-freeness gives the analogue of Proposition 4.2: if all nodes lie in S_M and B triples are returned, the call costs $O(M^3B)$ elementary operations and $O(MB)$ ring operations for the q-monomials.

Method 7.4 (Quantum `schubmult`)**.** Construct the strict completion and its label graphs. The coefficient ring is $\mathbb{Z}[q][y,z]$. Initialize $H(u, \mathrm{id}) = 1$. At column i put $k = \tilde{\theta}_i$ and create a fresh table H'.

(1) Group the keys of H by their first component w. For each such w let p_w be the maximum of $k - b$ over all label edges, of length b, leaving labels paired with w. Call `qpieri`(w, p_w, k) once.

(2) For each table entry $H(w,a)$, each edge $P : a \to a'$ of length b, and each returned triple $(w', t, q^{\delta'})$ with $t + b \le k$, accumulate

$$H'(w',a') \mathrel{+}= H(w,a)\,\varepsilon(P)\,q^{\delta'}\,E_{k-t-b,k-t}\big(Y(w,w';k); Z(P)\big).$$

(3) Replace H by H'. After column L, return $H(w, \tilde{a}_*)$; its coefficient of q^δ is $c^{w,\delta}_{u,v}(y;z)$.

In the ordinary quantum specialization keep only $t + b = k$.

Proposition 7.5. *Method 7.4 computes* (7.1)*.*

Proof. Multiply (7.4) by $\widetilde{\mathfrak{S}}_u(x;y)$ and apply (7.5) at each column, with $p' = k - b$. The coefficients in (7.5) involve only q, y, z, so they commute with the remaining factors. The proof of Theorem 4.5 applies verbatim, and no contribution is lost since $p' \le p_w$. □

7.4. **States of the quantum computation.** Write $D(w) = r_w$ for the last descent. Put

$$\rho = \max(r_u, \tilde{h}), \qquad s(d) = \max\{j \ge 0 : j(j+1) \le d\}, \qquad R_q = \min\big(M-1,\ \rho + \min(L, s(d))\big),$$

and

$$U_q = \min\left\{(M)_{\underline{R_q}}, \binom{d+R_q}{R_q}\right\}, \qquad M = n + L.$$

Lemma 7.6 (Quantum descent steps)**.** *Let t_{ab} be a quantum cover of w with $a \le k < b$. If it is a Bruhat cover then $D(wt_{ab}) \le \max(D(w), k)$. If it is length-decreasing then $b \le D(w) + 1$ and $D(wt_{ab}) \le \max(D(w), b)$. In particular, a single quantum cover raises D by at most one.*

Proof. Put $w' = wt_{ab}$ and $D = D(w)$. The entries of w after position D increase. Only positions a and b change, so a descent of w' after $\max(D, k)$ can only occur at $b-1$ or b with $b > D$.

For a Bruhat cover, $w'(b) = w(a) < w(b) < w(b+1)$, so there is no descent at b. If $b - 1 = a$ there is no descent at $b-1$ because $w'(a) = w(b) > w(a)$. Otherwise $w(b-1) < w(b)$, and the cover condition forces $w(b-1) < w(a) = w'(b)$.

For a length-decreasing cover, suppose $b \ge D + 2$. If $a < b - 1$ then $w(b-1)$ lies between $w(b)$ and $w(a)$, so $w(b-1) > w(b)$; but $b - 1 > D$. If $a = b - 1$ then $a > D$ and $w(a) < w(b)$. Either case is a contradiction, so $b \le D + 1$. The entries after position b are unchanged and increasing, so $D(w') \le \max(D, b)$. □

Lemma 7.7 (Quantum state bounds)**.** *Every permutation produced by a* `qpieri` *call in Method 7.4 lies in S_M, has length at most d, and has no descent after R_q. There are at most U_q such permutations. Every monomial q^δ that occurs involves only $q_1, \dots, q_{M-1}$ and has $|\delta| \le d/2$.*

Proof. Ambient size. Let $w \in S_N$ and $k < N$. For $c > N$, $w(c) = c$. Hence a first transposition with $b \ge N + 2$ is not a quantum cover: the value $N+1$ at position $N+1$ lies strictly between $w(a)$ and $w(b) = b$, which is forbidden for a Bruhat cover, and $w(a) < w(b)$ rules out a length-decreasing cover. Since the values b weakly decrease, the call stays in S_{N+1}. At column i the height is $k \le \tilde{h} \le n - 1$, so induction gives $S_{n+i} \subseteq S_M$. In particular, every quantum factor $q_a \cdots q_{b-1}$ has $b \le M$.

Length and q-degree. Classical covers add one to ℓ; a length-decreasing cover t_{ab} changes ℓ by $1 - 2(b-a)$ and multiplies by a monomial of degree $b - a$. Hence, along any chain of covers starting at u,

$$\ell(w) + 2|\delta| = \ell(u) + T, \tag{7.6}$$

where T is the total number of transpositions. Consider a sequence of pairs reaching a key (w, a) at boundary i, with chain lengths t_j and label edge lengths b_j. Then $t_j \le \tilde{\theta}_j - b_j$, and the labels are length-additive, so

$$T = \sum_{j \le i} t_j \le |\tilde{\theta}_{\le i}| - \ell(a), \qquad |\tilde{\theta}_{\le i}| = \sum_{j \le i} \tilde{\theta}_j.$$

The label a lies on a complete label path to $\tilde{a}_*$, whose later edges have total length at most $\sum_{j>i} \tilde{\theta}_j$. Lemma 7.2 gives $\ell(\tilde{a}_*) = |\tilde{\theta}| - \ell(v)$, so $\ell(a) \ge |\tilde{\theta}_{\le i}| - \ell(v)$ and $T \le \ell(v)$. A node produced from w in column $i+1$ has $t \le p_w = k - b$ for an edge $a \to a'$ leaving a label paired with w. Concatenating with a sequence reaching (w, a) gives $T + t \le |\tilde{\theta}_{\le i+1}| - \ell(a') \le \ell(v)$, by the same argument at boundary $i+1$. By (7.6) that node has length at most d, and the monomial along this sequence has degree at most $d/2$. All monomials attached to the node come from such sequences.

Descents. Fix such a sequence ending at a node w. All heights satisfy $k \le \tilde{h} \le \rho$, and $D(u) \le \rho$. By Lemma 7.6, a cover applied to a permutation with $D \le \rho + j - 1$ produces one with $D \le \rho + j$, and the bound $D \le \rho + j$ is reached from $D = \rho + j - 1$, for $j \ge 1$, only by a length-decreasing cover with $b = \rho + j$. Since $a \le k \le \rho$, that cover has q-degree $b - a \ge j$. If $D(w) = \rho + e$ with $e > 0$, the sequence therefore contains covers of q-degrees at least $1, 2, \dots, e$, so $e(e+1)/2 \le |\delta| \le d/2$. Furthermore, suppose a call first raises $\max(D, \rho)$ to $\rho + j$. This happens through a cover with $b = \rho + j$. All later covers in that call have $b \le \rho + j$, and by Lemma 7.6 they keep $D \le \rho + j$. Hence each call raises $\max(D, \rho)$ by at most one. A sequence through column i makes $i \le L$ calls. This proves $D(w) \le R_q$; the bound $D(w) \le M - 1$ is automatic.

The count U_q follows as in Lemma 5.2. □

Proof of Theorem 1.2. Correctness is Proposition 7.5. The cost is counted exactly as in the proof of Theorem 1.1, with U, V, h, θ replaced by $U_q, \widetilde{V}, \tilde{h}, \tilde{\theta}$, using Lemmas 7.2 and 7.7 and the strict label bound above. The table has at most $U_q\widetilde{V}$ keys because monomials in q are part of the coefficient. The ambient and degree assertions are in Lemma 7.7. For the coarse bound, use $L \le \tilde{h} \le n-1$, $M \le 2n-1$, $U_q \le (2n)^{R_q}$, $\widetilde{V} \le n^{\tilde{h}}$, $Q_{\tilde{h}}(n) \le 2n^{\tilde{h}}$, and $R_q \le \rho + L \le 2\rho$. The final claims follow from (7.2). □

7.5. **Remarks on the quantum bound.** *Dependence on L.* Classically $h = r_v$, so fixing code lengths suffices. In the quantum algorithm the factor $\tilde{h}$ also requires L to be bounded. This is not a weakness of the state estimate alone: the example $\mathfrak{S}_v = x_1^m$ has $r_v = 1$ and $\tilde{h} = m$. The difference comes from the distinct-height condition in Theorem 7.3. Both L and $\tilde{h}$ are computed from the input code of v, and $L \le \min(\ell(v), n-1)$. For ordinary quantum multiplication one may interchange u and v.

The worst-case parameters are pessimistic. For $u = v = (m+1)12\cdots m$, $2 \le m \le 8$, the measured numbers of distinct u-side states per column were m^2+m+1, and the label sets had m elements. The corresponding bounds U_q and $\widetilde{V}$ grow superpolynomially. We do not know whether polynomial bounds in n, for fixed code lengths, hold for the strict label graphs.

The implemented medium shape. The package's fast quantum kernels use the medium shape $\bar{\theta}$ of [22, Definition 4.1], which allows certain pairs of equal heights and has $\bar{\theta}_1 \le \tilde{h}$ [22, Proposition 4.3]. Pairs are processed by the doubled Pieri rule [22, Methods 4.14–4.15]. The termwise-quantization argument also applies to these shapes. Write $X_{k+1} = X_k \cup \{x_{k+1}\}$. Splitting $e_c(X_{k+1}) = e_c(X_k) + x_{k+1}e_{c-1}(X_k)$ and reindexing proves

$$e_a(X_k)e_b(X_k) = \sum_{c \le b} e_c(X_{k+1})e_{a+b-c}(X_k) - \sum_{c > a} e_c(X_{k+1})e_{a+b-c}(X_k).$$

In a medium shape the column before a pair of height k has height at least $k+2$, so the heights $k, k+1$ are not used elsewhere. Thus $\mathcal{Q}$ maps each filtered medium summand to the product of its single quantized factors and its doubled factors [22, (4.1)]. This gives a filtered version of [22, Theorem 4.7]. A doubled step must, however, retain the intermediate permutation of its two chains, so the same accounting gives a bound with U_q^3 in place of U_q^2. We do not analyze the implementation's additional degree cap.

Prior work. Quantum Schubert polynomials [3], the quantum Pieri rule [19], and equivariant quantum Schubert calculus [12, 8] give algorithms for these coefficients. We are not aware of a previous explicit bound for computing an entire quantum product on complete flag varieties in terms of input parameters. Since the q^0 part of (7.1) is (1.1), the hardness remarks in Section 1.5 apply to it.

8. Implementation and computational checks

The package's ordinary and double kernels use the same main data flow, as Proposition 6.1 predicts: compute the dominant code of v^{-1}, construct backward label graphs, enumerate Pieri endpoints once per first-component state, and accumulate matched contributions. Methods 3.4 and 4.4 follow the classical algorithm of [22, Methods 4.9–4.11]. In particular, the package's `kdown_perms` restriction to positions still holding their original value is the distinct-position condition justified by Lemma 3.2. The Cauchy filter in Lemma 2.3 separately explains which column endpoints may be passed to the next layer.

The remaining distinction concerns arithmetic evaluation. The package's elementary-weight routine uses an unmemoized divide-and-conquer expression construction. The circuit implementation analyzed in Theorem 1.1 uses the shared recurrence (4.2). For a literal recursive implementation, a conservative upper bound is obtained by multiplying (1.3) by $(h+1)^{O(\log(h+1))}$. This leaves the general $2^{O(n \log n)}$ bound and fixed-code-length polynomiality unchanged, but is not the same operation count.

Arithmetic-circuit complexity is also not a wall-clock bound for a computer algebra system that canonicalizes expressions, copies subtrees, or performs automatic simplification. Probabilistic removal of zero coefficients is not part of the algorithm analyzed here. Theorems 1.1 and 1.2 apply to the explicitly specified algorithms: the triple Method 4.4 and the strict-shape quantum Method 7.4, together with their double and ordinary specializations. They do not cover Grothendieck kernels, the independent transition–Monk kernel, the medium-shape quantum kernel with its doubled Pieri steps (see Section 7), merged-column implementations, or every released optimization. The package's reference quantum routines use the strict shape and single-column steps, as Method 7.4 does.

8.1. **Parabolic quantum output.** The parabolic quantum ring computes a complete-flag quantum product and then applies the Peterson–Woodward comparison term by term. The correctness of this reduction, including the triple coefficients, is proved in [22, Section 9]. The implementation does not construct a lift from a requested parabolic degree. Instead, it splits each complete-flag coefficient into its q-monomials, tests whether each exponent vector is a Peterson–Woodward lift, and, for an accepted monomial, multiplies the indexing permutation by the two parabolic longest elements, tests minimality, and reindexes the surviving quantum variables.

This gives a direct runtime accounting for the projection. Let N be the ambient rank used by the projection, let C be the number of permutation–q-monomial pairs presented to it, and let F be the work of splitting the coefficient expressions by their q-monomials. For each such pair, the exponent tests and reindexing take $O(N^2)$ elementary operations. The current `check_blocks` implementation checks all subinterval sums by nested loops, and the longest-element routine constructs the relevant elements by adjacent swaps. Each therefore costs at most $O(N^3)$ operations. Permutation multiplication, the minimality test, and the ambient-rank test cost at most $O(N^2)$ in this bound. Consequently, the projection costs

$$O(F + N^3 C).$$

Here F is kept separate because `factor_out_q` recursively decomposes symbolic expressions and convolves the resulting monomial maps at multiplication nodes; its cost is not one operation per surviving monomial.

In the path-counting model, each distinct permutation–q-monomial pair in the complete-flag output has at least one path contribution, so C is bounded by the work of the complete-flag computation. Thus the comparison adds at most a polynomial-in-N factor to that path-count bound, in addition to the explicit symbolic q-splitting work F. This is a postprocessing bound for the implemented reduction, not a faster multiplication algorithm: the complete-flag product is still computed first, and the projection does not reduce its state space.

8.2. **Computational checks.** The specified update was compared with the ordinary Python kernel on all 616 pairs from S_2, S_3, S_4. The double update was checked on all pairs from S_3 at three integer specializations and on 40 additional pairs from S_4, both against the double Python kernel and using independent divided-difference evaluation of the Schubert polynomials on both sides of (1.1). The dominant-completion and descent assertions were also checked through S_6, and the boundary counts on 528 weak intervals through S_5. The label graphs and their coefficient alphabets were compared with the implementation on all 148 nonidentity inputs from S_2 through S_5. The distinct-position enumeration was compared with `kdown_perms`, and with the unrestricted weighted path sum, on 2728 column instances through S_5. The elementary recurrence was checked in all 91 degrees $0 \le p \le m \le 12$, including repeated coefficient parameters. For Corollary 6.3, the rectangular completion and the Grassmannian property of every enumerated u-side state were checked on 4233 pairs of Grassmannian permutations through S_7.

For the quantum section, Method 7.4 was implemented independently, with `qpieri` written from the definition of a quantum cover and compared with `elem_sym_perms_q` on every call. Its ordinary specialization agreed with both package quantum kernels, the strict-shape reference and the medium-shape fast kernel, on all 584 pairs with $v \neq \mathrm{id}$ from S_2, S_3, S_4 and on 149 random pairs from S_5. The triple coefficients agreed with both quantum double kernels at random integer specializations of y, z, q on all 30 such pairs from S_3 and 60 random pairs from S_4. At $z = y$ the computed coefficients satisfied $c_{u,v}^{w,\delta} = c_{v,u}^{w,\delta}$ on 555 pairs through S_4. In all these runs the assertions of Lemma 7.7 and the label bound $|\tilde{A}_i| \le \widetilde{V}$ were checked for every enumerated state. Lemma 7.2 was checked on all 5906 nonidentity permutations through S_7.

9. Further questions

Several refinements of the input-parameter bounds remain. One is to replace $Q_h(n)$ by a smaller count exploiting the Bruhat-cover conditions, beyond the distinct-position restriction already justified by [22]. For Grassmannian inputs this is the only obstruction to a $2^{O(n)}$ bound in Corollary 6.3. Another is to exploit the degree cap and the actual dominant column heights simultaneously, instead of replacing all column heights by h.

In the quantum case the main question is whether the dependence on L is necessary. The example x_1^m in Section 7 has quadratically many u-side states and linearly many labels, while Theorem 1.2 allows exponentially many. A bound polynomial in n for fixed r_u, r_v would require either a quantization argument

tolerating repeated column heights, such as the doubled factors of the medium shape, or a direct count of strict label graphs. A sharper bound on the descent excess in Lemma 7.7 would also help; in random tests through S_9 it never exceeded 2.

The fixed-code-length bound also suggests a practical division of labor. Explicit multiplication can generate whole tables for moderate numerical code entries, while the bounded-range lattice-point method of Pak and Slonim is designed for compressed single-coefficient inputs. No asymptotic crossover or universal practical superiority follows from the two available upper bounds alone.

Appendix A. Using the conjectural Grothendieck interface

The package provides `Gx` and `DGx` for ordinary and double Grothendieck polynomials, and `QDGx` for quantum double Grothendieck polynomials. Multiplication in `QDGx` uses a conjectural equivariant quantum K-Pieri rule. Its non-equivariant specialization at $\beta = -1$ is the Pieri rule of Naito and Sagaki [14]. The Grothendieck kernels are outside the scope of the complexity bounds proved in this paper.

A.1. **Python interface and conventions.** The constructor `QDGx(w, beta=-1)` accepts a permutation in one-line notation and uses the coefficient alphabet y. Its second argument selects another alphabet: `QDGx(w, z, beta=-1)` uses z, while `QDGx(w, "0", beta=-1)` gives the non-equivariant version. Omitting `beta` leaves the deformation parameter symbolic; $\beta = -1$ is the usual K-theory convention. The double Grothendieck convention uses $a \oplus b = a + b + \beta ab$. Consequently, at $\beta = 0$ the quantum double basis specializes to $\mathfrak{S}^q_w(x; -y)$, with the opposite coefficient alphabet from the Schubert convention used in the main text.

Elements support linear combinations, multiplication, and coefficient access by `Permutation` keys. The method `simplify()` cancels rational coefficients while retaining the quantum Grothendieck basis; `expand()` instead returns the explicit x-polynomial using Lenart–Maeno quantization [10]. For example:

```
>>> from schubmult import QDGx, Permutation
>>> from schubmult.abc import z
>>> g = QDGx([2, 1], "0", beta=-1)
>>> g.expand()
q_1 + x_1 - x_1*q_1
>>> p = (g * g).simplify()
>>> for w in sorted(p):
...     print(w, p[w])
() q_1
(1, 3, 2) -q_1
(3, 1, 2) 1
```

Here `()` indexes the identity. These are stable polynomial products: input permutations in S_n do not impose a finite-flag quotient, so output labels may lie outside S_n. Conversion of arbitrary x-polynomials to the quantum Grothendieck basis and coproducts are not implemented by `QDGx`.

Independent coefficient alphabets give quantum triple products. Both factors must use the same value of β; the result is in the left factor's coefficient alphabet. We print the coefficient dictionary to keep the rational expressions readable:

```
>>> g = QDGx([2, 1], beta=-1)
>>> h = QDGx([2, 1], z, beta=-1)
>>> p = (g * h).simplify()
>>> for w in sorted(p):
...     print(w, p[w])
() -q_1*(-1 + z_1)/(1 - y_2)
(1, 3, 2) q_1*(-1 + z_1)/(1 - y_2)
(2, 1) (-y_2 + z_1)/(1 - y_2)
(3, 1, 2) (1 - z_1)/(1 - y_2)
```

Specializing $z = y$ gives the equivariant product, and setting both coefficient alphabets to zero gives the non-equivariant product.

A.2. **Multiplicative coefficient variables.** At $\beta = -1$, `ChangeOfVariables` can replace the default coefficient coordinates by $1 - y_i^{-1}$, so that the variables y_i are multiplicative torus characters:

```
>>> from schubmult import ChangeOfVariables, QuantumDoubleGrothendieckRing
>>> from schubmult.abc import x, y
>>> characters = ChangeOfVariables(z, y, lambda i, t: 1 - 1/t[i])
>>> R = QuantumDoubleGrothendieckRing(x, characters, beta=-1)
>>> r = (R([2, 1])**2).simplify()
>>> r[Permutation([])]
y_2*q_1/y_1
>>> r[Permutation([2, 1])]
(y_1 - y_2)/y_1
```

In these coordinates the non-equivariant specialization is $y_i = 1$, rather than $y_i = 0$. No change of quantum parameters is required.

A.3. **Finite flags and parabolic projection.** For parabolic quantum K-theory, the Python interface applies `apply_kato` to the complete-flag product dictionary. At $\beta = -1$ this implements Kato's quotient map [5]: it replaces a permutation by its minimal right-coset representative and sets the quantum parameters belonging to the parabolic subgroup to 1. Unlike the command-line block sizes, the Python argument lists the *1-indexed simple reflections* generating that subgroup. The argument `n` specifies the ambient S_n; terms indexed outside S_n are discarded, and the surviving quantum parameters are reindexed consecutively. An empty generator list with an explicit `n` gives the finite complete-flag restriction.

For blocks $(2, 2)$, the generators are $[1, 3]$ and $n = 4$. The complete-flag parameter q_2 becomes the parabolic parameter q_1. Here is the mixed-alphabet analogue of the Grassmannian example in Section 1.1:

```
>>> from schubmult.mult.groth_quantum_double import apply_kato
>>> a = QDGx([3, 4, 1, 2], beta=-1)
>>> b = QDGx([1, 3, 2], z, beta=-1)
>>> coefficients = apply_kato(a * b, [1, 3], n=4, beta=-1)
>>> parabolic = a.ring.from_dict(coefficients).simplify()
>>> for w in sorted(parabolic):
...     print(w, parabolic[w])
(1, 3, 2) q_1*(-1 + z_1)*(-1 + z_2)/((1 - y_3)*(1 - y_4))
(3, 4, 1, 2) (-y_3 - y_4 + z_1 + z_2 + y_4*y_3 - z_1*z_2)/((1 - y_3)*(1 - y_4))
```

The inputs are minimal coset representatives. Here `from_dict` is used only for simplification and display, not to construct a parabolic ring. Use the same β for multiplication and projection. Since individual terms may contain inverse powers of β, a $\beta = 0$ specialization must follow symbolic projection and coefficient cancellation.

A.4. **Command-line usage.** The four programs are:

`grothmult_py`	Ordinary Grothendieck products
`grothmult_double`	Double and mixed products
`grothmult_q`	Non-equivariant quantum products
`grothmult_q_double`	Equivariant and mixed quantum products

All accept `--beta` to specialize the deformation parameter, and `--code` to use Lehmer codes. The double programs accept `--mixed-var` for independent coefficient alphabets and `--simplify` to cancel coefficients. The quantum examples above can be computed as follows:

```
$ grothmult_q 2 1 - 2 1 --beta=-1
()  q_1
(1, 3, 2)  -q_1
(3, 1, 2)  1
$ grothmult_q_double 2 1 - 2 1 --mixed-var --beta=-1 --simplify
       ()  (q_1 - z_1*q_1)/(1 - y_2)
(1, 3, 2)  (-q_1 + z_1*q_1)/(1 - y_2)
   (2, 1)  (-y_2 + z_1)/(1 - y_2)
(3, 1, 2)  (1 - z_1)/(1 - y_2)
```

Parabolic inputs must be minimal coset representatives. The quantum programs take *block sizes* after `--parabolic` and handle the delayed $\beta = 0$ specialization automatically.

The option `--multiplicative-variables` selects $\beta = -1$ and the coordinates $1 - y_i^{-1}$ for either double program, also using $1 - z_i^{-1}$ when `--mixed-var` is present. Thus the parabolic triple product above, in multiplicative coordinates, is:

```
$ grothmult_q_double 3 4 1 2 - 1 3 2 --parabolic 2 2 \
>     --mixed-var --multiplicative-variables --simplify
   (1, 3, 2)  y_4*y_3*q_1/(z_1*z_2)
(3, 4, 1, 2)  (-y_4*y_3 + z_1*z_2)/(z_1*z_2)
```

The quantum double command does not currently implement `--display-positive`.

References

[1] Buch, A. S. Littlewood–Richardson Calculator. Software package, version 2.1, including the program `schubmult`; `https://sites.math.rutgers.edu/~asbuch/lrcalc/`. First released 1999.

[2] Ciocan-Fontanine, I. On quantum cohomology rings of partial flag varieties. *Duke Mathematical Journal 98*, 3 (1999), 485–524.

[3] Fomin, S., Gelfand, S., and Postnikov, A. Quantum Schubert polynomials. *Journal of the American Mathematical Society* (1997), 565–596.

[4] Hwang, B.-H. Coefficient lifting for skew divided differences and triple Schubert coefficients, 2026. arXiv:2610.21053, version 1.

[5] Kato, S. On quantum K-groups of partial flag manifolds, 2019. arXiv:1906.09343.

[6] Kim, B. On equivariant quantum cohomology. *International Mathematics Research Notices*, 17 (1996), 841–851.

[7] Kirillov, A. N., and Maeno, T. Quantum double Schubert polynomials, quantum Schubert polynomials and Vafa–Intriligator formula. *Discrete Mathematics 217*, 1-3 (2000), 191–223.

[8] Lam, T., and Shimozono, M. Quantum double Schubert polynomials represent Schubert classes. *Proceedings of the American Mathematical Society 142*, 3 (2014), 835–850.

[9] Lascoux, A., and Schützenberger, M. Schubert Polynomials and the Littlewood-Richardson Rule. *Letters in Math. Physics 10* (1985), 111–124.

[10] Lenart, C., and Maeno, T. Quantum Grothendieck polynomials, 2006. arXiv:math/0608232.

[11] Macdonald, I. G. *Notes on Schubert Polynomials*. Université Du Québec à Montréal, Dép. de mathématiques et d'informatique, 1991.

[12] Mihalcea, L. C. Equivariant quantum Schubert calculus. *Advances in Mathematics 203*, 1 (2006), 1–33.

[13] Monk, D. The geometry of flag manifolds. *Proc. London Math. Soc. 9* (1959), 253–286.

[14] Naito, S., and Sagaki, D. Pieri-type multiplication formula for quantum Grothendieck polynomials. *Advances in Mathematics 459* (2024). arXiv:2211.01578.

[15] Narayanan, H. On the complexity of computing Kostka numbers and Littlewood-Richardson coefficients. *J. Algebraic Combin. 24*, 3 (2006), 347–354.

[16] Pak, I., and Robichaux, C. Vanishing of Schubert coefficients. *arXiv preprint arXiv:2412.02064* (2024). with an appendix joint with David E. Speyer.

[17] Pak, I., and Robichaux, C. Vanishing of Schubert coefficients in probabilistic polynomial time, 2025. arXiv:2509.16467.

[18] Pak, I., and Slonim, Z. Stretched Schubert coefficients are eventually quasi-polynomial, 2026. arXiv:2604.27107, version 1.

[19] Postnikov, A. On a quantum version of Pieri's formula. In *Advances in geometry*. Springer, 1999, pp. 371–383.

[20] Samuel, M. J. A Molev-Sagan type formula for double Schubert polynomials. *J. Pure Appl. Algebra 228*, 7 (2024), 107636.

[21] Samuel, M. J. `schubmult`: a Python package for multiplying Schubert polynomials. Available from the Python Package Index, `https://pypi.org/project/schubmult/`, with sources at `https://github.com/matthematics/schubmult`, 2026.

[22] Samuel, M. J. Triple quantum Schubert calculus, 2026. arXiv:2610.31533; with an appendix by Tongren Xiao and Yihua Yue; version 1.

[23] Sottile, F. Pieri's rule for flag manifolds and Schubert polynomials. *Ann. Inst. Fourier 46*, 1 (1996), 89–110.

[24] The Sage Developers. *SageMath, the Sage Mathematics Software System*. `https://www.sagemath.org/`.

Email address: `matthematics@gmail.com`